

UNIT III

Trees: Binary tree, Terminology, Representation, Traversals, Applications – Binary search tree – AVL tree.

B Trees: B Tree indexing, operations on a B Tree, Lower and upper bounds of a B Tree - B + Tree Indexing – Trie Tree Indexing.

2 MARKS

1. Define tree?

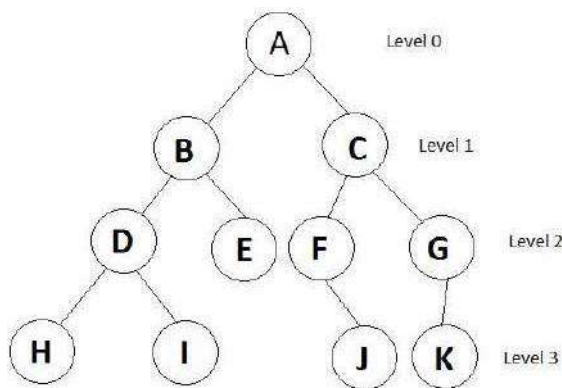
Tree is a non linear data structure. There is no linear relation between the data items.

It can be defined as finite set of more than one node.

There is a special node designated as root node.

The remaining nodes are partitioned into sub tree of a tree.

The below diagram shows the tree



2. Define a path in a tree

The path in a tree is referred as the nodes in which the successive nodes are connected by the edge in a tree For eg: the path from A to I

A-B, B-D, D-I.

3. Define terminal nodes in a tree

A node that has no children is called as a terminal node. It is also referred as a leaf node. These nodes have degree has zero.

4. Define non-terminal nodes in a tree?

All intermediate nodes that traverse the given tree from its root node to the terminal nodes are referred as non terminal nodes.

5. Define branch, siblings & ancestors?

Branch or edge of a tree is called as the link or connection between two nodes. The nodes having the same parent are called siblings. The ancestor of a node is referred as all nodes along the path of root node to the path node.

6. What is an AVL tree? (Nov 11)

An AVL tree (Adelson-Velskii and Landis' tree, named after the inventors) is a self-balancing binary search tree.

7. Define B tree? (Nov 12, Apr 13, Nov 13)

B-tree is a tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic time

8. State the properties of tree?

Any node can be the root of the tree.

If the root is identified; then that tree is called as the rooted node.

If it is not identified; then that tree is called as the free tree.

Every node, except the root node has a unique parent.

9. Define degree?

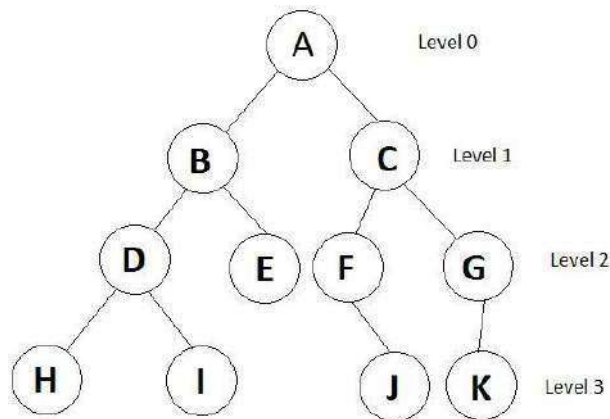
The degree of a node is referred as the number of sub-trees of a particular node.

10. Define height of the tree?

The height or depth of a tree is defined as maximum level of any node in a tree.

11. Define a binary tree? (Nov 11, Nov 12, Apr 13)

A binary tree is tree, which has nodes either empty or not more than two child nodes each of which may be a leaf node.



12. Define a full binary tree?

A full binary tree is a tree in which all the leaves are on the same level and every non-leaf node has exactly two children.

13. Define a complete binary tree?

A complete binary tree is a tree in which every non-leaf node has exactly two children not necessarily to be on the same level.

14. Define a right skewed binary tree?

A right skewed binary tree is a tree, which has only right child nodes

15. State the properties of a binary tree.

The maximum number of nodes on level n of the binary tree is 2^{n-1} where $n \geq 1$

The maximum number of nodes in a binary tree of height n is $2^n - 1$ where $n \geq 1$

For any non empty tree $n_l = n_d + 1$ where n_l is the number of leaf nodes and n_d is the number of nodes of degree 2.

16. What are the different ways of representing a binary tree?

Linear representation using arrays

Linked representation using pointers

17. What is meant by binary tree traversal?

Traversing a binary tree means moving through all the nodes in the binary tree visiting each node in the tree only once.

18. What are the difference binary tree traversal techniques?

Preorder

Inorder

Postorder

19. What are the tasks performed while traversing a binary tree?

Visiting a node

Traverse the left sub tree

Traverse the right subtree

20. What are the tasks performed during preorder traversal?

Process the root node.

Traverse the left subtree

Traverse the right subtree

21. What are the tasks performed during inorder traversal?

Traverse the left subtree

Process the root node.

Traverse the right subtree

22. What are the tasks performed during postorder traversal?

Traverse the left subtree

Traverse the right subtree

Process the root node.

23. State the merits and demerits of linear representation of binary trees

Merits:

Storage method is easy and can be easily implemented in arrays

When the location of a parent /child node is known other one can be determined easily.

It requires static memory allocation so it is easily implemented in all programming language

Demerits:

Insertions and deletions in a node, take an excessive amount of processing time due to data movement up and down the array.

24. State the merits and demerits of linked representation of a binary tree

Merits:

Insertions and deletions in a node, involve no data movement except the rearrangement of pointers, hence less processing time.

Demerits:

Given a node structure, it is difficult to determine its parent node.

Memory spaces are wasted for storing null pointers for the nodes, which have one or no subtrees.

It requires dynamic memory allocation, which is not possible in some programming languages.

25. Define a binary search tree.

A binary search tree is a special binary tree which is either empty or if it is not empty it should satisfy the following characteristics.

Every node has a value and no two nodes should have the same value.

The values in any left subtree are less than the value of its parent node.

The values in any right subtree are greater than the values of its parent node.

The left and right subtrees of each node are again binary search trees.

26. What do you mean by general trees?

General tree is a tree with nodes having any number of children

27. Define Trie tree. (Nov 13)

A trie, also called digital tree and sometimes radix tree or prefix tree (as they can be searched by prefixes), is an ordered tree data structure that is used to store a dynamic set or associative array where the keys are usually strings.

28. What is the length of the path in a tree? (May 14)

The path length of a tree is the sum of the levels of all the tree's nodes.

The path length of a tree with N nodes is the sum of the path lengths of the subtrees of its root plus N-1.

29. Define B+ tree indexing. (May 14)

A B+ tree can be viewed as a B-tree in which each node contains only keys (not key-value pairs), and to which an additional level is added at the bottom with linked leaves. The primary value of a B+ tree is in storing data for efficient retrieval in a block-oriented storage context — in particular, file systems.

30. What are the applications of binary tree? (Nov 14)

- | | | | | |
|-----------------------|------------------------|---------------|----------|-----------|
| 1. Binary search tree | 2. Binary tries | 3. Hash trees | 4. Heaps | 5. T-tree |
| 6. Syntax tree | 7. Huffman coding tree | | | |

31. What is the use of indexing technique? (Nov 14)

Indexing is an auxiliary data structure which speeds up the record retrieval.

32. Compare B-tree and B+ tree (Nov 14)

- A B+ tree is the form of a balanced tree in which every path from the root of the tree to a leaf of the tree is the same length. B+ trees are good for searches but cause some overhead issues in wasted space.
- B-trees are similar to B+ trees but it allows search-key values to appear only once, eliminates redundant storage of search keys. It is possible sometimes to find search-key value before reaching the leaf node. Implementation is harder than B+ trees.

33. What is the maximum number of nodes in a binary tree of depth d? (Nov 14)

The maximum no of nodes in a tree of depth 'D' is $2^{(d+1)} - 1$.

34. Differentiate between binary tree and binary search tree (Apr 15)

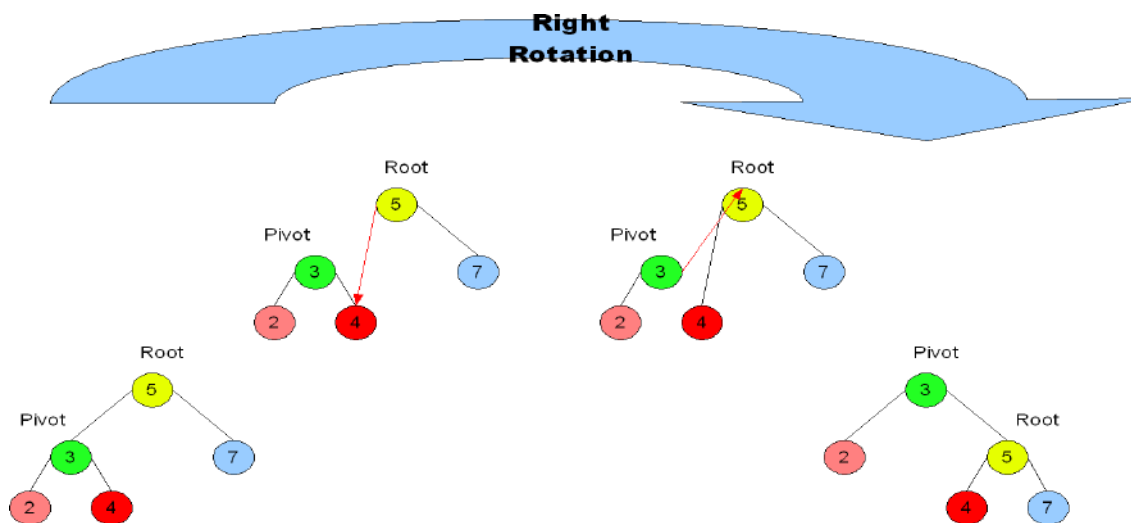
- Binary is a specialized form of tree with two child (left child and right child). It is simply representation of data in tree structure.
- Binary search tree is a special type of binary tree that follows the following condition:
 - Left child is smaller than its parent node
 - Right child is greater than its parent node

35. Give some of the applications of B tree (Apr 15)

B tree is optimized for systems that read and write large blocks of data. It is commonly used in databases and file systems.

B tree also optimizes costly disk accesses that are of concern when dealing with large data sets.

36. Perform RR rotation in AVL tree with an example. (Apr 15)



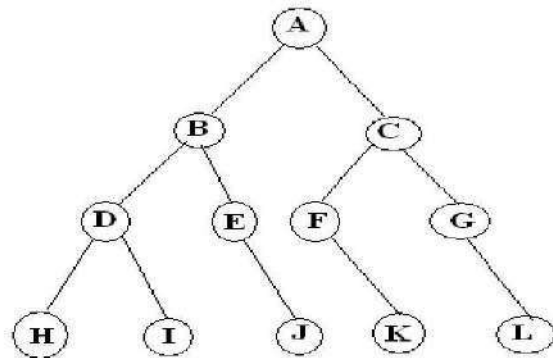
11 MARKS

1. Explain about the terminologies of Trees?

Tree is a non linear data structure. There is no linear relation between the data items.

It can be defined as finite set of more than one node.

The below diagram shows the tree



⇒ There is a special node designated as root node.

The remaining nodes are partitioned into sub tree of a tree. The root node does not have the parent relation

The remains has a parent associated to it .The node may not have a child node. It is called as leaf node or terminal node.

The intermediate nodes travels from root node to the leaf node are called non terminal nodes. Branch or edge is referred as the link or connection between the two nodes.

The path in a tree is referred as the nodes in which the successive nodes are connected by the edge in a tree

For eg: the path from A to
| A-B, B-D, D-I.

The level of the tree, the root node is treated as LEVEL

0. The nodes have a same parent is called siblings.

The height or depth of a tree is defined as maximum level of any node in a tree.

The degree of a node is defined as the number of sub tree in a particular node

The ancestor of the node is defined as all the nodes along the path from the root node to the particular node. For eg: the ancestor of the node H is

D, B, A.



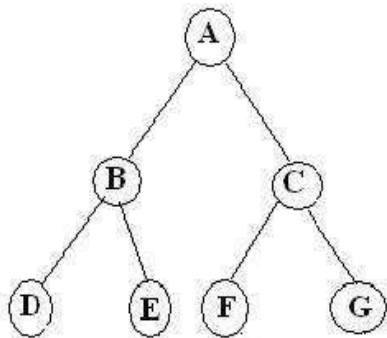
PROPERTIES OF THE TREE:

Any node can be the root of the tree

Every node has a property there is exactly one path connect the root node to the particular node. In a tree ,the root is identified is called rooted tree .If the root is not identified is called free tree Every node except the root node have the unique parent

BINARY TREE:

Binary tree is defined which is either empty or consists of a root node and disjoint sub tree called left child and right child. The below diagram shows the binary tree.

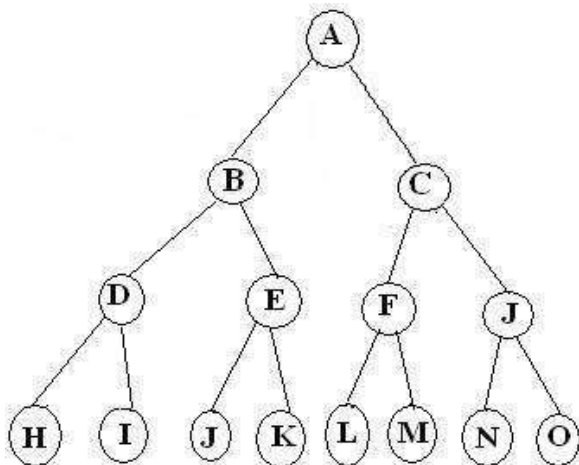


In a binary tree no node can have more than two children. Each

binary tree is a tree, but not every tree is a binary tree.

The complete binary tree is defined as the intermediate nodes have a degree and leaf nodes are at the same level.

The below diagram shows the complete binary tree



PROPERTIES OF BINARY TREE :

The maximum number of nodes in the tree of level l is 2^l where l is greater than or equal to 0.

The maximum no of nodes in a tree of depth 'D' is $2^{(d+1)} - 1$.



APPLICATION OF BINARY TREE :

1. Solving 2 way decision problem.
2. binary tree traversal



SOLVING TWO WAY DECISION PROBLEM :

2 ways decision problem is a problem in which we have to do a lot of comparisons to get the results.

Consider ,

For eg:

The removal of the duplicate
number Consider the numbers

25, 10, 25, 40, 17, 8, 10,

25 ○ **METHOD 1:**

We can remove the duplicate numbers, by comparing the each number with another numbers in the list. In this method ,we need lot of comparisons

○ **METHOD 2:**

The number of comparisons can be reduced by representing the numbers in a binary tree

IMPLEMENTATION:

Take the first number in the list is treated as root with empty left and right child.

Each successive number in the list is compared with the root value and to the following steps

- a) If it is equal to root value then remove it or if it is greater do step (b)

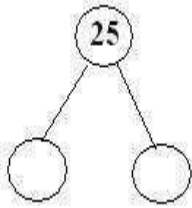
Else do step (c)

- b) Check the right child is empty or not ,if the right child is empty place it as right child (or)compare with the existing right child and repeat step(a)
- c) Check the left child is empty or not, if it is empty, place it as left child or compare with the existing left child and repeat step (a).

Repeat step 2 till the last number of the list.

The nos. are 25, 10, 25, 40, 17, 8, 10, 25

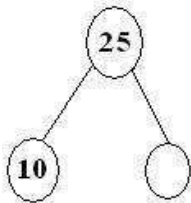
STEP 1:



STEP 2 :

$10 < 25$

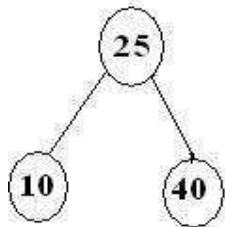
So step (c) is implemented



STEP 3:

$25 = 25$, so it is removed,

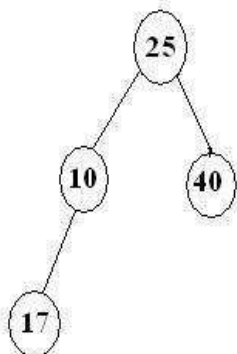
$40 < 25$, so step (b) is implemented



STEP 4:

$17 < 25$

So step (c) is implemented



2. Explain about the Binary tree traversal? (Nov 11, Nov 12, Apr 13, May 14, Nov 14, May 15)

BINARY TREE TRAVERSALS

- ❖ A binary tree traversal requires that each node of the tree be processed once and only once in a predetermined sequence.
- ❖ The two general approaches to the traversal sequence are,
 - ***Depth first traversal***
 - ***Breadth first traversal***
- ❖ In depth first traversal, the processing proceeds along a path from the root through one child to the most distant descendent of that first child before processing a second child. *In other words, in the depth first traversal, all the descendants of a child are processed before going to the next child.*
- ❖ In a breadth-first traversal, the processing proceeds horizontally from the root to all its children, then to its children's children, and so forth until all nodes have been processed. *In other words, in breadth traversal, each level is completely processed before the next level is started.*

Depth-First Traversal

There are basically three ways of binary tree traversals. They are :

- 1. Pre Order Traversal**
- 2. In Order Traversal**
- 3. Post Order Traversal**

In C, each node is defined as a structure of the following form:

```
struct node
{
    int info;
    struct node *lchild;
    struct node *rchild;
}

typedef struct node NODE;
```

Binary Tree Traversals (Recursive procedure)

1. Inorder Traversal

- Steps :
1. Traverse left subtree in inorder
 2. Process root node
 3. Traverse right subtree in inorder

Algorithm

C Coding

```
void inorder_traversal ( NODE * T)
```

Algorithm inorder traversal (Bin-Tree T)

Begin

If (not empty (T)) then

Begin

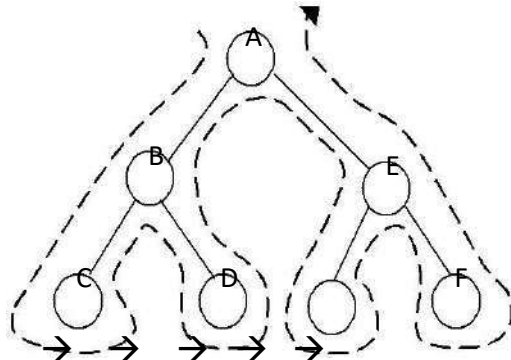
Inorder_traversal (left subtree (T))

Print (info (T)) /* process node */

Inorder_traversal (right subtree (T))

End

End



The Output is : C B D A E F G

2. Preorder Traversal

Steps : 1. Process root node

2. Traverse left subtree in preorder

3. Traverse right subtree in preorder

Algorithm

Algorithm preorder traversal (Bin-Tree T)

Begin

If (not empty (T)) then

Begin

Print (info (T)) /* process node */

Preoder traversal (left subtree (T))

Inorder traversal (right subtree (T))

End

End

}

if(T != NULL)

{

inorder_traversal(T->lchild);

printf("%d \t ", T->info);

inorder_traversal(T->rchild);

}

}

C function

void preorder_traversal (NODE *T)

{

if(T != NULL)

{

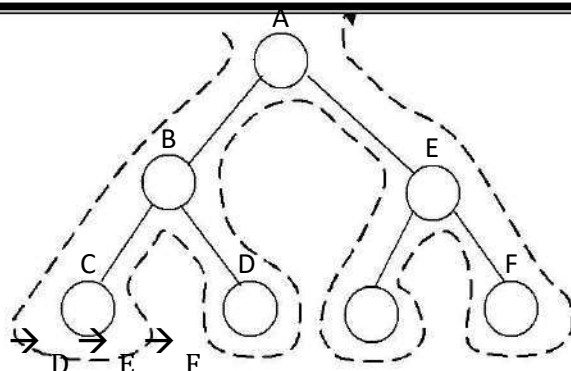
printf("%d \t", T->info);

preorder_traversal(T->lchild);

preorder_traversal(T->rchild);

}

}



Output is : A → B → C → D → E → F

3. Postorder Traversal

- Steps :
1. Traverse left subtree in postorder
 2. Traverse right subtree in postorder
 3. process root node

Algorithm

Postorder Traversal

Algorithm postorder traversal (Bin-Tree T)

Begin

If (not empty (T)) then

Begin

Postorder_traversal (left subtree (T))

Postorder_traversal (right subtree(T))

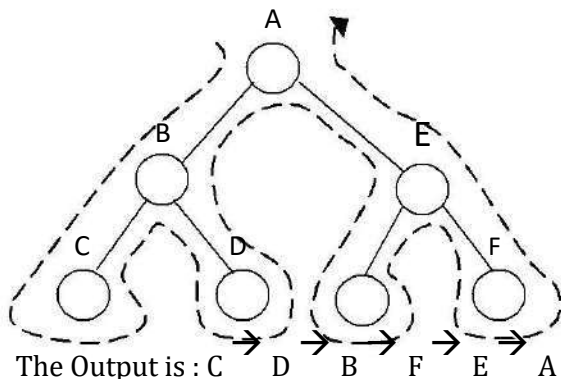
Print (Info (T)) /* process node */

End

End

C function

```
void postorder_traversal ( NODE * T)
{
    if( T != NULL)
    {
        postorder_traversal(T->lchild);
        postorder_traversal(T->rchild);
        printf("%d \t", T->info);
    }
}
```



3. Write in detail about the application of trees?

SET REPRESENTATION

The tree is used in representation of set .the elements of the set are the numbers from 1 to n .consider the disjoint set S_i, S_j where $i \neq j$

That means there is no element is both in S_i, S_j .

Let us consider the Numbers 1 to 10 is partition into 3 disjoint subsets S_1, S_2, S_3

$S_1 = 1, 7, 8, 9.$

$S_2 = 4, 5, 10$

$S_3 = 2, 3, 6.$

In set we can perform various operations like union and find.

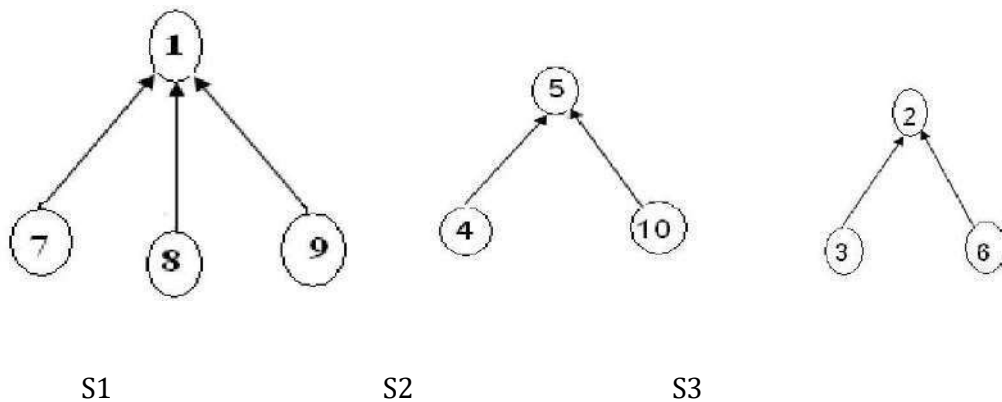
Union of set S_1 and S_2 is such that

$$S_1 \cup S_2 = \{1, 4, 5, 7, 8, 9, 10\}$$

All the elements X which is in S_1 and S_2

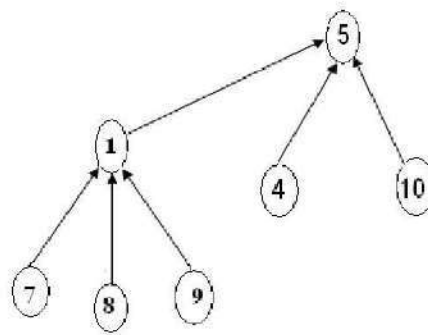
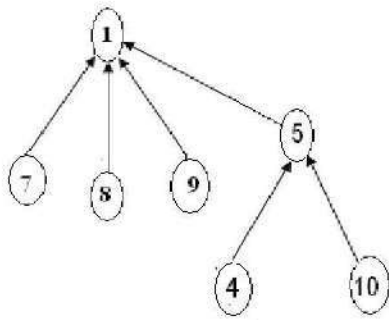
Find the algorithm used to find the set containing the element i.

The representation of sets using tree is shown below

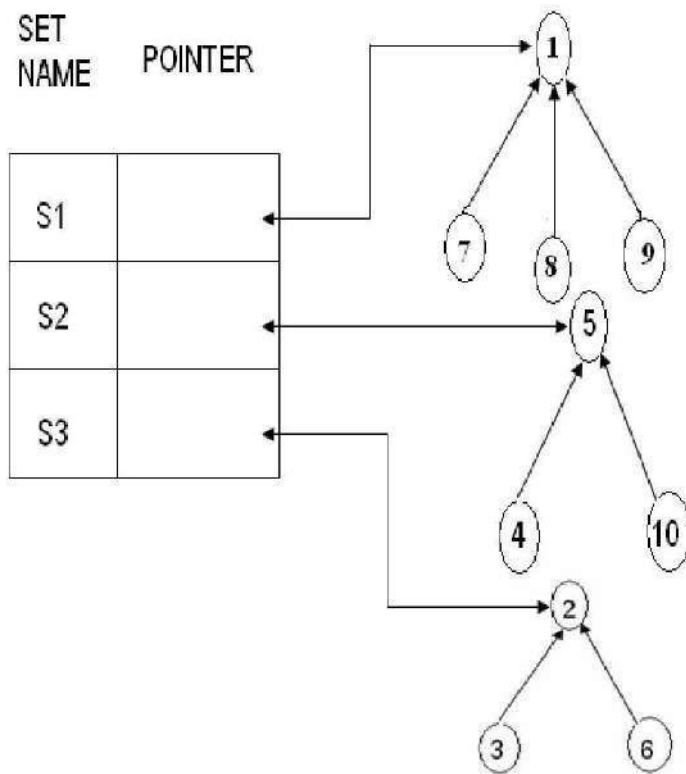


In the above diagram, all the nodes are linked on the parent relationship except the root node; all other nodes are linked to the parent.

$S_1 \cup S_2$ is represented by 2 possible representations.



The data representation of set is shown below



⇒ The data representation of set is mainly used for find algorithm

Find algorithm determine the element I with root node j has a entry k into the set name table

ALGORITHM :

UNION(I,J)

\\ To replace the two root node i,j where i is not equal to j with their union

PARENT (I) J ←

END OF UNION

FIND(I)

\\ To find the root node j of the element i

```

J    I
WHILE PARENT (I) > 0.
DO
  J = 0, if this node is root node
  PARENT (I) = J
END
RETURN J
END

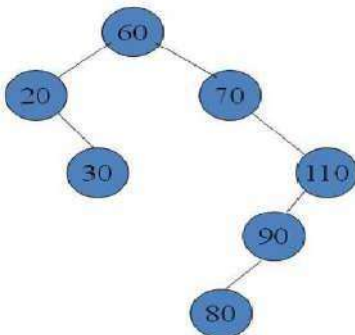
```

4. Explain about the AVL trees? (Apr 12, May 15)

- Consider a situation when data elements are inserted in a BST in sorted order: 1, 2, 3, ...
- BST becomes a degenerate tree.
- Search operation
FindKey takes $O(n)$,
which is as inefficient as in a list.
- It is possible that after a number of insert and delete operations a binary tree may become imbalanced and increase in height.
- Can we insert and delete elements from BST so that its height is guaranteed to be $O(\log n)$? → Yes,
AVL Tree ensures this.
- Named after its two inventors: Adelson-Velski and Landis.

Imbalanced Tree:

An Imbalanced Tree

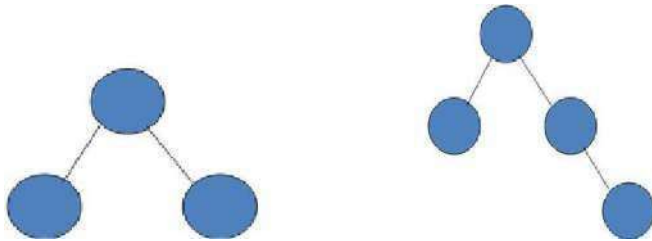


A Balanced Tree

AVL Tree: Definition

- Height-balanced tree: A binary tree is a height-balanced-p-tree if for each node in the tree, the difference in height of its two subtrees is at the most p.
- AVL tree is a BST that is height-balanced-1-tree.

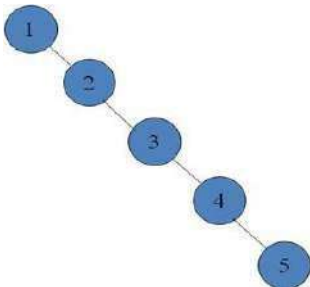
AVL Trees: Examples



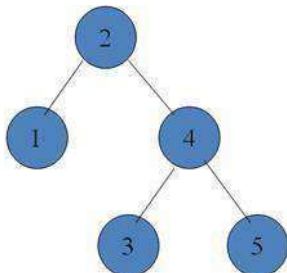
AVL Trees

Inserting 1, 2, 3, 4 and 5

BST after insertions



AVL Tree after insertions



ADT AVL Tree:

Elements: The elements are nodes, each node contains the following data type: Type

Structure: Same as for the BST; in addition the height difference of the two subtrees of any node is at the most one.

Domain: the number of nodes in a AVL is bounded; type AVLTree

ADT AVL Tree

Operations:

1. **Procedure** FindKey (AVLTree AVL, KeyType tkey, boolean found).
2. **Procedure** Insert (AVLTree AVL, Type e, boolean inserted).
3. **Procedure** Remove_Key (AVLTree AVL, KeyType tkey, boolean deleted)
4. **Procedure** Update(AVLTree AVL, Type e)

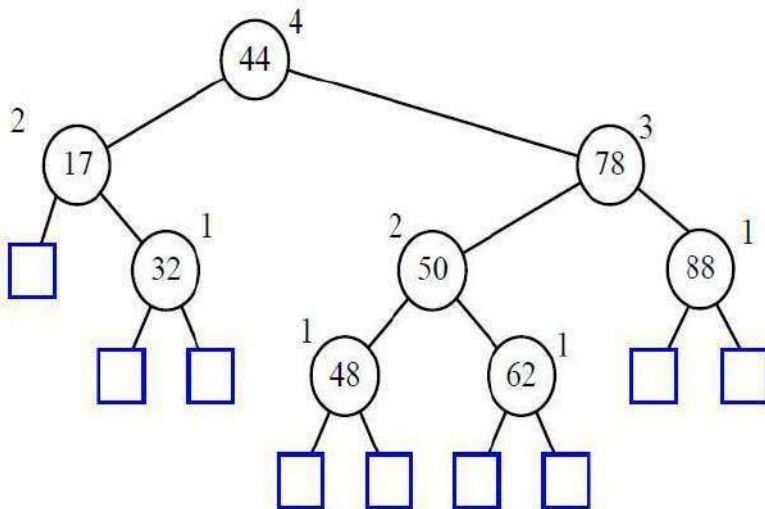
5. **Procedure** Traverse (AVLTree AVL, Order ord)
6. **Procedure** DeleteSub (AVLTree AVL)
7. **Procedure** Retrieve (AVLTree AVL, Type e)
8. **Procedure** Empty (AVLTree AVL, boolean empty).
9. **Procedure** Full (AVLTree AVL, boolean full)

AVL Tree: Insert

- Step 1: A node is first inserted into the tree as in a BST.
- There is always a unique path from the root to the new node called the search path.
- Step 2: Nodes in the search path are examined to see if there is a pivot node. Three cases arise.
- A pivot node is a node closest to the new node on the search path, whose balance is either -1 or $+1$.
- Case 1: There is no pivot node. No adjustment required.
- Case 2: The pivot node exists and the subtree to which the new node is added has smaller height. No adjustment required.
- Case 3: The pivot node exists and the subtree to which the new node is added has the larger height. Adjustment required.

AVL Tree

- An AVL Tree is a binary search tree such that for every internal node v of T , the heights of the children of v can differ by at most 1.
- An example of an AVL tree where the heights are shown next to the nodes

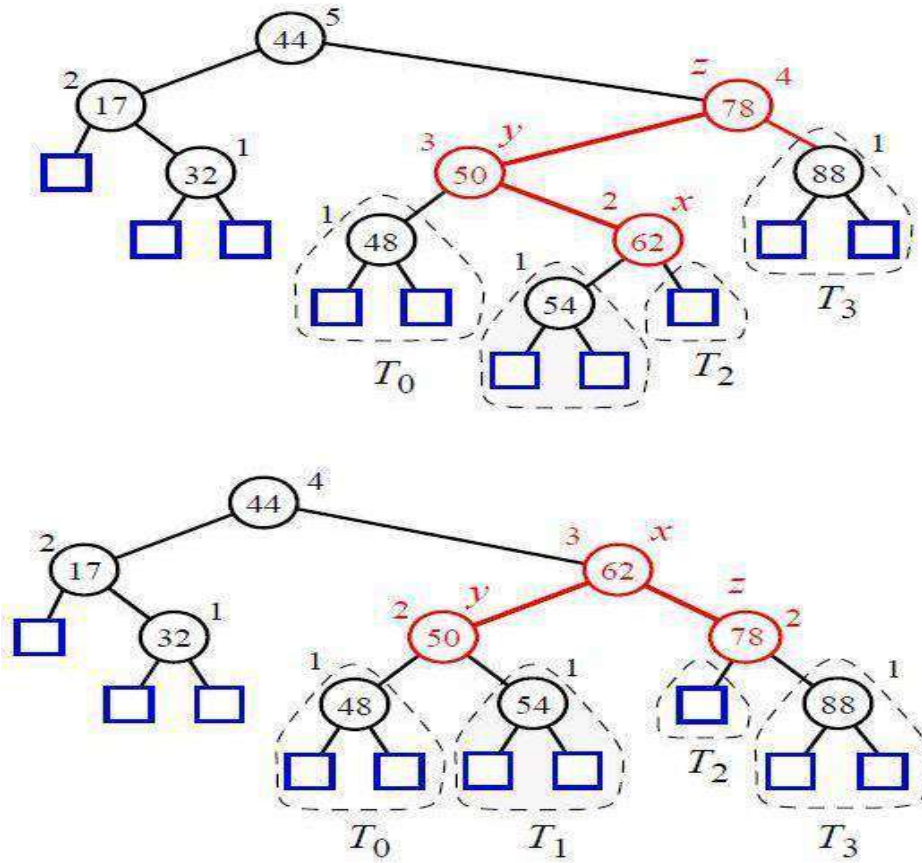


Insertion

- A binary search tree T is called balanced if for every node v , the height of v 's children differ by at most one.
- Inserting a node into an AVL tree involves performing an `expandExternal(w)` on T , which changes the heights of some of the nodes in T .
- If an insertion causes T to become unbalanced, we travel up the tree from the newly created node until we find the first node x such that its grandparent z is an unbalanced node.

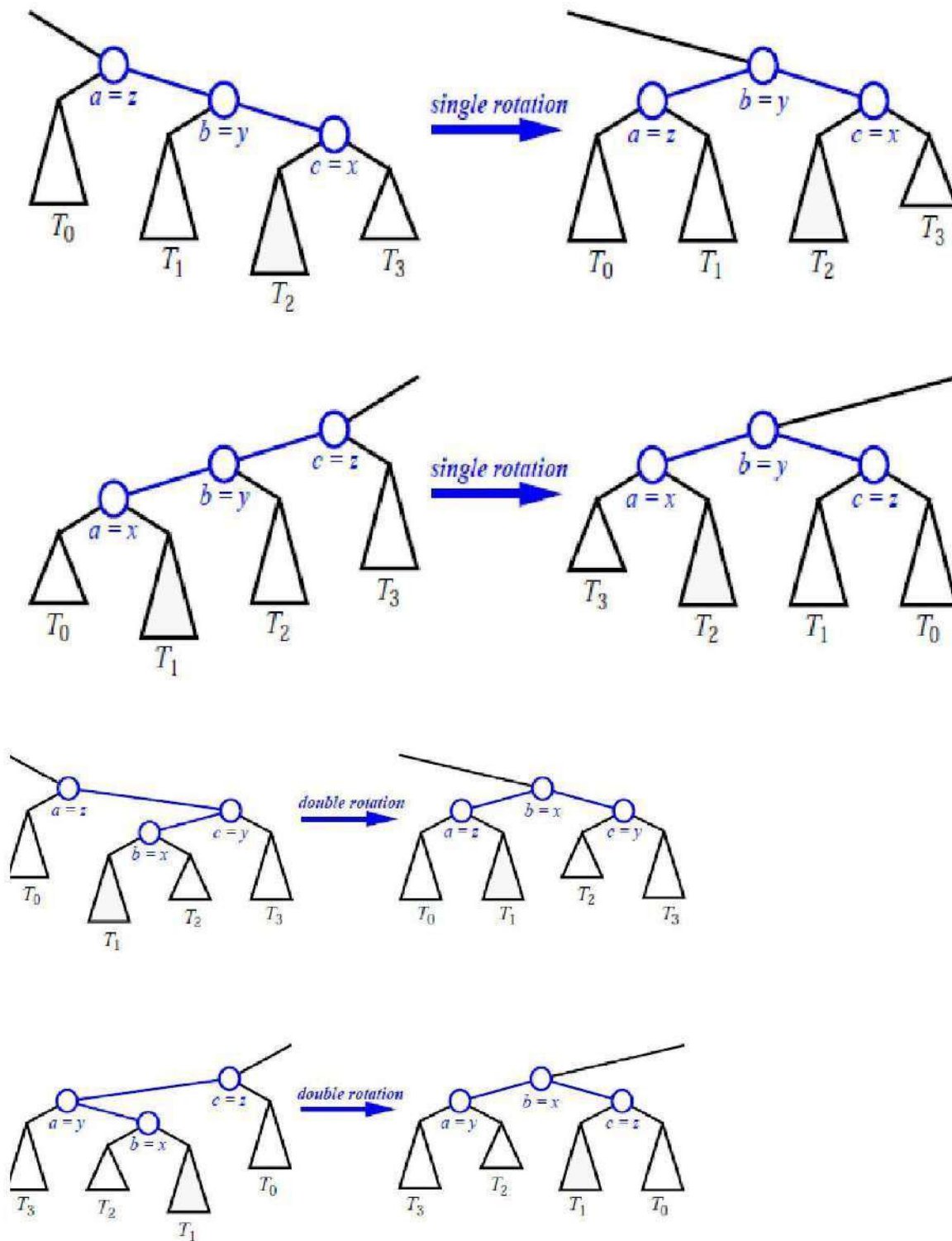
- Since z became unbalanced by an insertion in the subtree rooted at its child y , $\text{height}(y) = \text{height}(\text{sibling}(y)) + 2$
- To rebalance the subtree rooted at z , we must perform a *restructuring*- we rename x , y , and z to a , b , and c based on the order of the nodes in an in-order traversal.- z is replaced by b , whose children are now a and c whose children, in turn, consist of the four other subtrees formerly children of x , y , and z .

Example of insertion into an AVL tree.



Restructuring

- The four ways to rotate nodes in an AVL tree, graphically represented:
- Single Rotations:

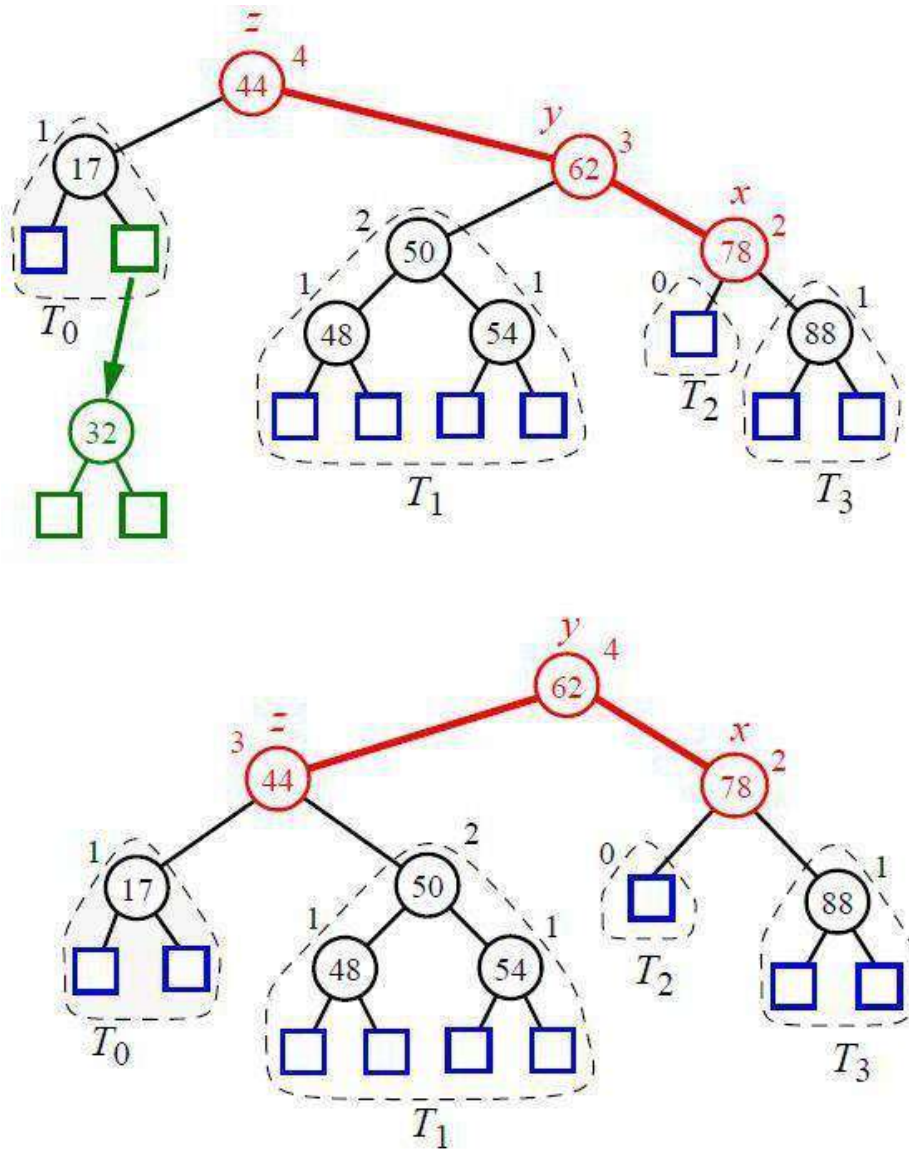


Removal

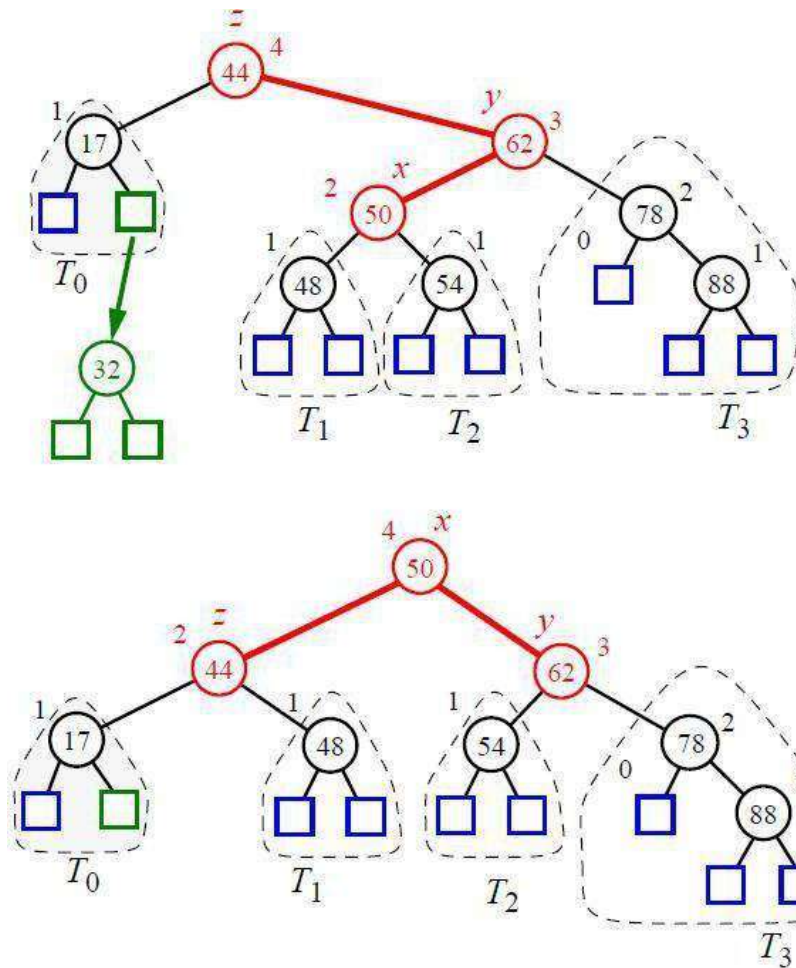
- We can easily see that performing a `removeAboveExternal(w)` can cause T to become unbalanced.
- Let z be the first unbalanced node encountered while travelling up the tree from w . Also, let y be the child of z with the larger height, and let x be the child of y with the larger height.
- We can perform operation `restructure(x)` to restore balance at the subtree rooted at z .

- As this restructuring may upset the balance of another node higher in the tree, we must continue checking for balance until the root of T is reached.

Example of deletion from an AVL tree:



Example of deletion from an AVL tree



5. Explain about the B-trees? (Apr 13) (Nov 12, May 14, Nov 14)

- B-trees are balanced search trees: height = $O(\log(n))$ for the worst case.
- They were designed to work well on Direct Access secondary storage devices (magnetic disks).
- Similar to red-black trees, but show better performance on disk I/O operations.

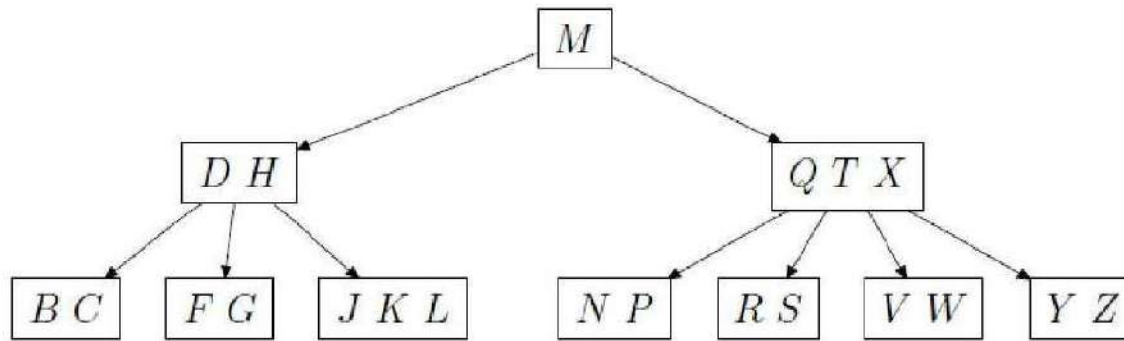
Data structures on secondary storage:

- Memory capacity in a computer system consists broadly on 2 parts:
 1. Primary memory: uses memory chips.
 2. Secondary storage: based on magnetic disks.
- Magnetic disks are cheaper and have higher capacity.
- But they are much slower because they have moving parts.

B-trees try to read as much information as possible in every disk access operation. B-trees (and variants like B+ and B* trees) are widely used in database systems.

An example

The 21 english consonants as keys of a B-tree:



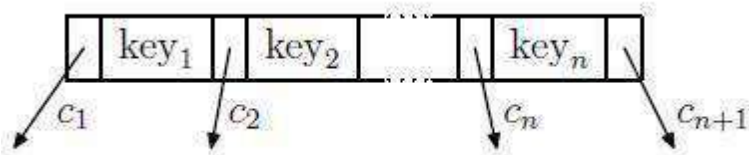
- Every internal node x containing $n[x]$ keys has $n[x] + 1$ children.
- All leaves are at the same depth in the tree.

B-tree: definition

A B-tree T is a rooted tree (with root $\text{root}[T]$) with properties:

- Every node x has four fields:
 1. The number of keys currently stored in node x , $n[x]$.
 2. The $n[x]$ keys themselves, stored in nondecreasing order:
 $\text{key1}[x] _ \text{key2}[x] _ \dots _ \text{keyn}[x][x]$.
 3. A boolean value,
 $\text{leaf}[x] =$
 (
 - True if x is a leaf ,
 - False if x is an internal node .
 4. $n[x] + 1$ pointers, $c1[x], c2[x], \dots, cn[x]+1[x]$ to its children.
 (As leaf nodes have no children their c_i are undefined).

- Representing pointers and keys in a node:



Basic operations on B-trees?

Details of the following operations:

- B-Tree-Search
- B-Tree-Create
- B-Tree-Insert
- B-Tree-Delete

Conventions:

- Root of B-tree is always in main memory (Disk-Read on the root is never required)
- Any node passed as parameter must have had a Disk-Read operation performed on them.

Procedures presented are all top down algorithms (no need to back up) starting at the root of the tree. Searching a B-tree

2 inputs: x , pointer to the root node of a subtree, k , a key to be searched in that subtree.

function B-Tree-Search(x, k) returns (y, i) such that $\text{key}_i[y] = k$ or nil

$i \leftarrow 1$

while $i \leq n[x]$ and $k > \text{key}_i[x]$

do $i \leftarrow i + 1$

if $i \leq n[x]$ and $k = \text{key}_i[x]$

then return (x, i)

if leaf[x]

then return nil

else Disk-Read($\text{ci}[x]$)

return B-Tree-Search($\text{ci}[x], k$)

At each internal node x we make an $(n[x] + 1)$ -way branching decision.

Creating an empty B-tree

B-Tree-Create(T)

$x \leftarrow \text{Allocate-Node}()$

leaf[x] $\leftarrow$ true

$n[x] \leftarrow 0$ Disk-

Write(x)

root[T] $\leftarrow x$

- Allocate-Node() allocates one disk page to be used as a new node
- requires $O(1)$ disk operations and $O(1)$ CPU time

Inserting a key into a B-tree (I)

- The key is always inserted in a leaf node
- Inserting is done in a single pass down the tree
- Requires $O(h) = O(\log_t n)$ disk accesses
- Requires $O(th) = O(t \log_t n)$ CPU time
- Uses B-Tree-Split-Child to guarantee that recursion never descends to a full node

Inserting a key into a B-tree

2 inputs: T , the root node,

k , key to insert.

```

B-Tree-Insert(T, k)
r root[T]
if n[r] = 2t - 1
then s Allocate-Node()
root[T] s
leaf[s] false
n[s] 0

```

Inserting a key into a nonfull node of a B-tree

```

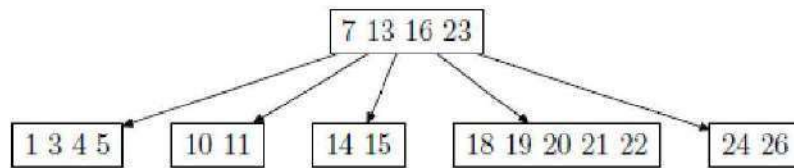
B-Tree-Insert-Nonfull(x, k)
i n[x]
if leaf[x]
then while i > 1 and k <
keyi[x] do keyi+1[x] keyi[x]
i i - 1
keyi+1[x] k
n[x] n[x] + 1
Disk-Write(x)
else while i > 1 and k <
keyi[x] do i i - 1
i i + 1 Disk-
Read(ci[x]) if n
ci[x] = 2t - 1
then B-Tree-Split-Child
x, i, ci[x]
if k > keyi[x]
then i i + 1
B-Tree-Insert-Nonfull(ci[x], k)
c1[s] r B-Tree-Split-
Child(s, i, r) B-Tree-Insert-
Nonfull(s, k) else B-Tree-
Insert-Nonfull(r, k)
Uses B-Tree-Insert-Nonfull to insert key k into nonfull node x

```

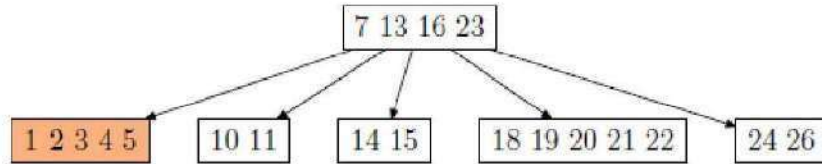
Inserting a key - Examples (I)

Initial tree:

$t = 3$

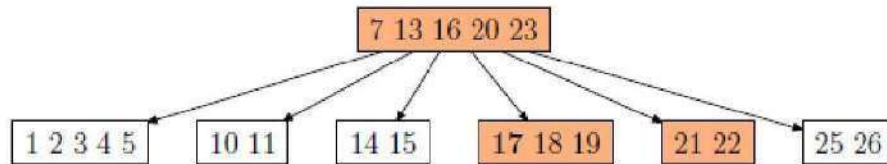


2 inserted:



17 inserted:

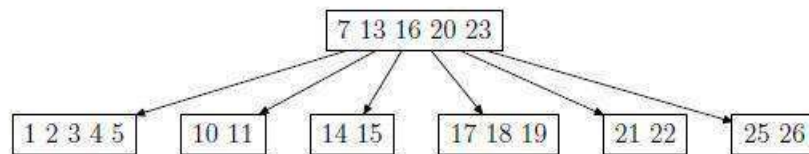
(to the previous one)



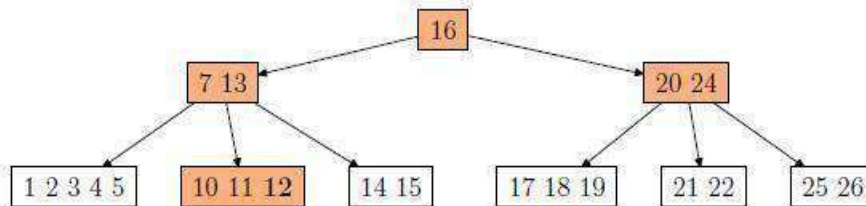
Inserting a key - Examples (II)

Initial tree:

$t = 3$

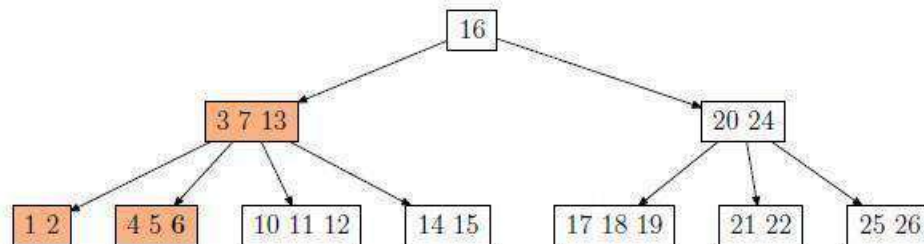


12 inserted:



6 inserted:

(to the previous one)

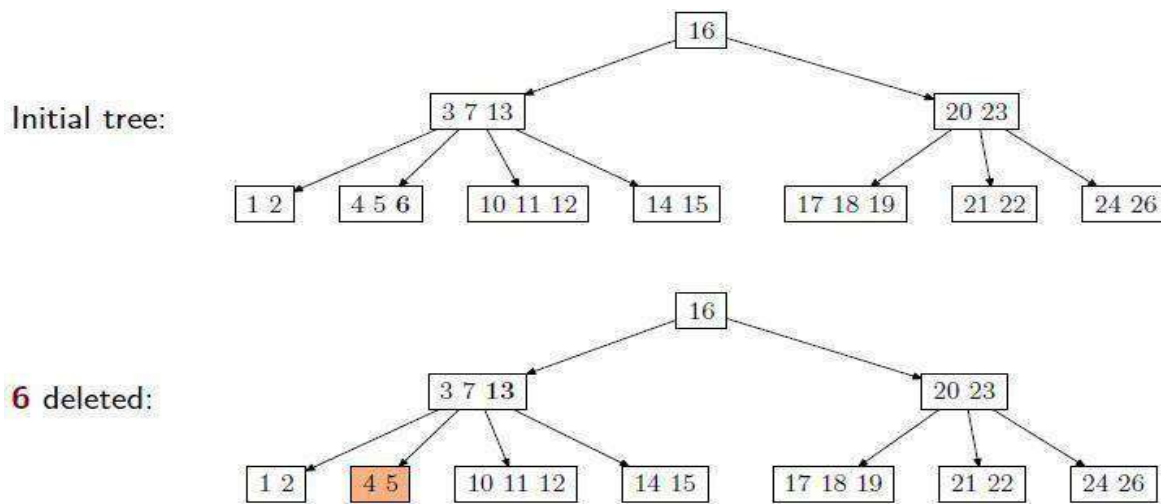


Deleting a Key from a B-tree

- Similar to insertion, with the addition of a couple of special cases
- Key can be deleted from any node.
- More complicated procedure, but similar performance figures: $O(h)$ disk accesses, $O(th) = O(t \log t n)$ CPU time

- Deleting is done in a single pass down the tree, but needs to return to the node with the deleted key if it is an internal node
- In the latter case, the key is first moved down to a leaf. Final deletion always takes place on a leaf

Deleting a Key — Case 1



- The first and simple case involves deleting the key from the leaf. $t - 1$ keys remain

6. Explain about the upper and lower bound of a Btree?

To determine the height of a B-Tree of order m containing n keys, we solve each of the above for h , as follows:

- a. From the equation for the maximum number of keys, we know:

$$n \leq m^h - 1$$

or, solving for h :

$$n+1 \leq m^h$$

$$\log(n+1) \leq hm$$

Now, since h must be an integer, we can take the ceiling of the log to obtain:

$$\lceil \log(n+1) \rceil \leq hm$$

- b. From the equation for the minimum number of keys, we know:

$$n \geq 2 * c^{(h-1)} - 1$$

or, solving for h

$$\frac{(n+1)}{2} \geq c^{h-1}$$

$$\log_c \left(\frac{(n+1)}{2} \right) \geq h-1$$

$$h \leq 1 + \log_c \left(\frac{(n+1)}{2} \right)$$

Now, since h must be an integer, we can use the floor of the log to obtain:

$$h \leq 1 + \left\lfloor \log_c \left(\frac{(n+1)}{2} \right) \right\rfloor \quad (\text{floor of the log to the base } c)$$

c. Combining the above results from minimal and maximal trees, we obtain the following bounds for h:

$$\left\lceil \log_c(n+1) \right\rceil \leq h \leq 1 + \left\lfloor \log_c \left(\frac{(n+1)}{2} \right) \right\rfloor$$

4. Some examples:

a. 1 million keys - B-Tree of order 200: height is 3

- Lower bound is $\left\lceil \log_{200} 1,000,001 \right\rceil = 3$

(Note that a maximal tree of height 2, order 200, contains 39,999 keys - so tree must have height at least 3)

- Upper bound is $1 + \text{floor}(\log_{100} 500,001) = 3$

(Note that a minimal tree of height 4, order 200, contains 1,999,999 keys, so tree must have height no greater than 3)

a. 2 million keys - B-Tree of order 200:

- Lower bound is still 3

- Upper bound is now 4

so the height could be 3 or 4.

7. Explain about the B+Trees? (Nov 11, Apr 12, Nov 14, May 14)

A B-tree is very efficient with respect to search and modification operations that involve a single record –For example, let $m = 100$ and $N = 1000000$; in such case the search of a key requires at most 4 disk accesses

A B-tree however is not particularly suited for sequential operations nor for range searches The retrieval of the next key value may requires accessing a large number of nodes

To address such problem a variation to the B-tree structure has been proposed, known as B+-tree

Main idea

In a B-tree, the key values have two functions: separatorso determine the path to follow during the search

Key values to allow accessing the information associated with them (that is, the pointers to the data)

In a B+-tree such functions are kept distinct:

The leaf nodes contain all the key values (and the associated information)

The internal nodes (organized as a B-tree) store some separators which have the only function of determining the path to follow when searching for a key value

In addition the leaf nodes are linked in a list, in order to efficiently support range searches or sequential searches (there is also a pointer to the first element of such list to support fast accesses to the minimum key value) partial duplication of the keys

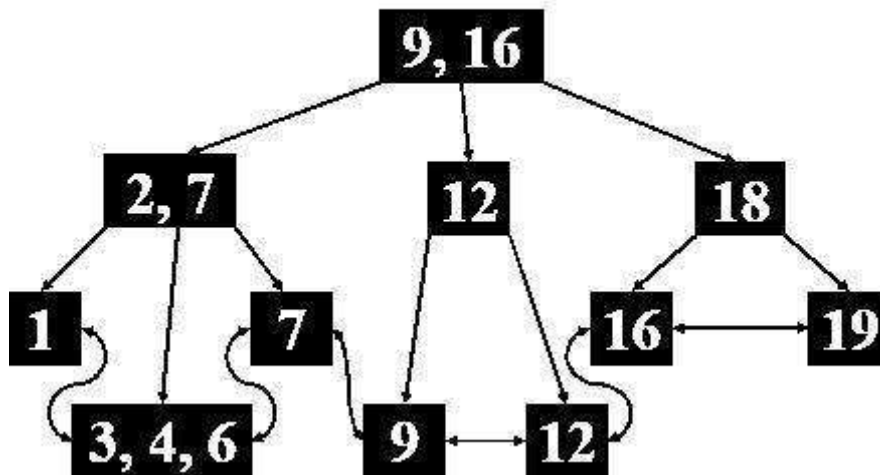
The index entries (keys and data references) are only stored in the leaf nodes

A search for a given key value must always determine a leaf node

The subtree on the left side of a separator contains key values that are lower than the separator; the subtree on the right side of a separator contains key values which are greater or equal than the separator

In the case of alphanumeric keys, one can reduce the space requirements by using separators that have reduced lengths

B+ Tree Example

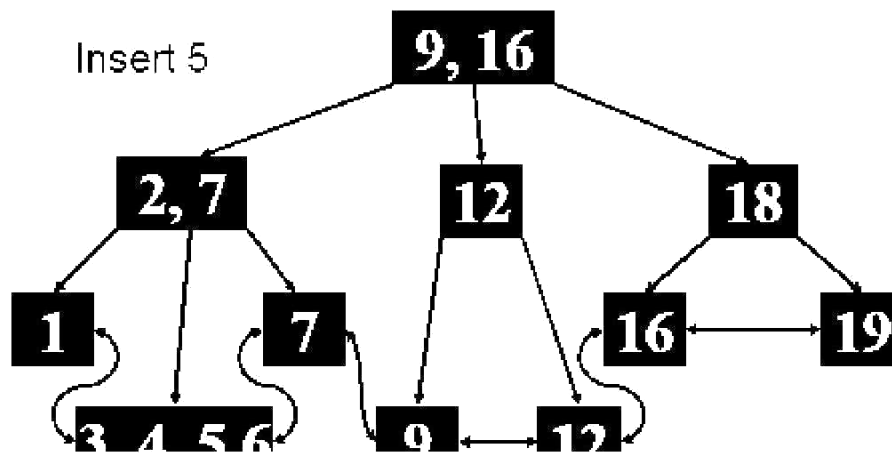


B+Tree insertion:

Insert at bottom level If leaf page overflows, split page and copy middle element to next index page

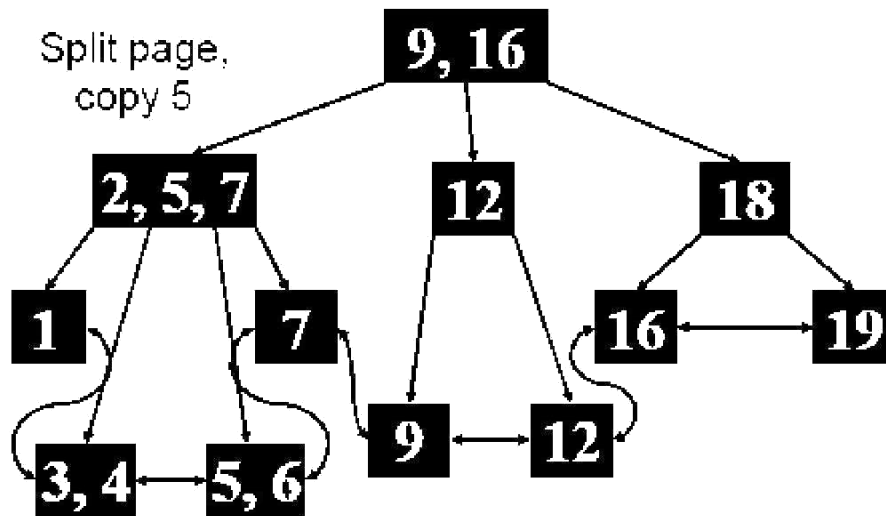
If index page overflows, split page and move middle element to next index page

Insert 5



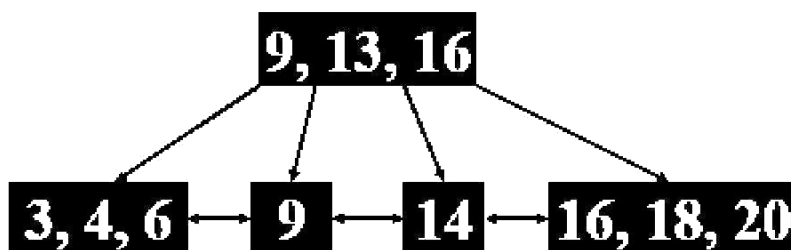
B+ Tree Insertion Example

Split page,
copy 5



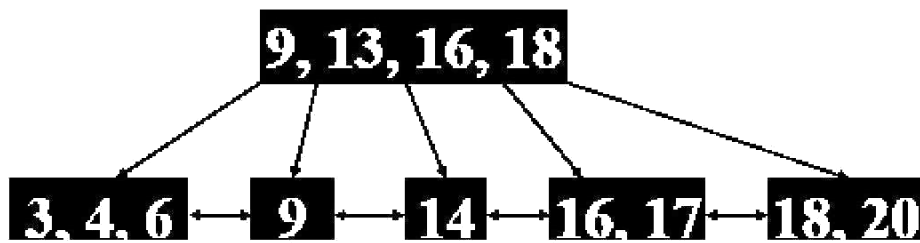
B- Tree Insertion Example 2

Insert 17



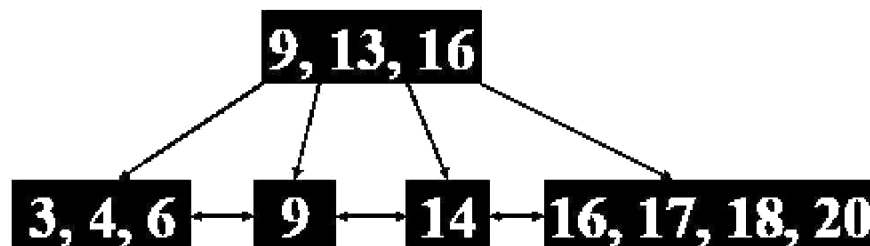
B- Tree Insertion Example 2

Split leaf
page, copy 18



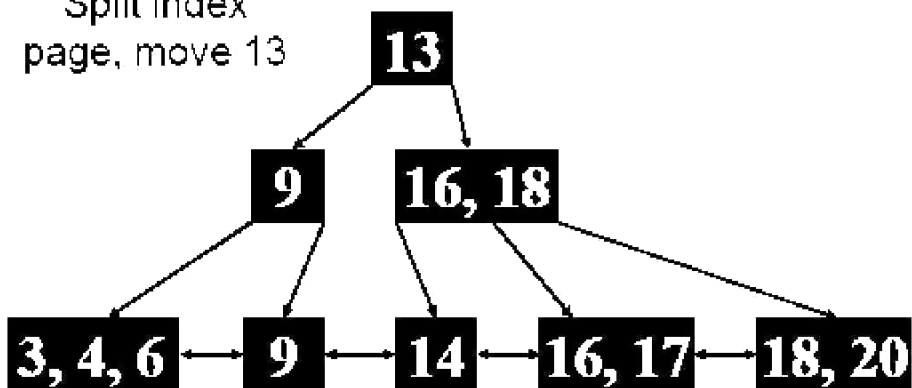
B+ Tree Insertion Example 2

Insert 17



B+ Tree Insertion Example 2

Split index
page, move 13

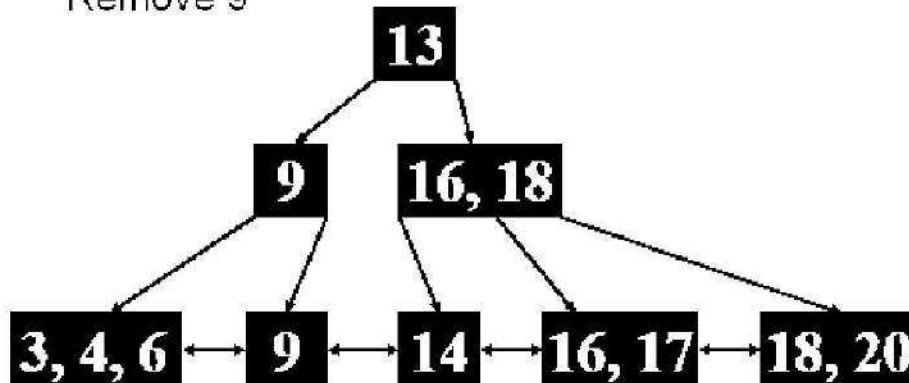


B+Tree Deletion:

- Delete key and data from leaf page
- If leaf page underflows, merge with sibling and delete key in between them
- If index page underflows, merge with sibling and move down key in between them

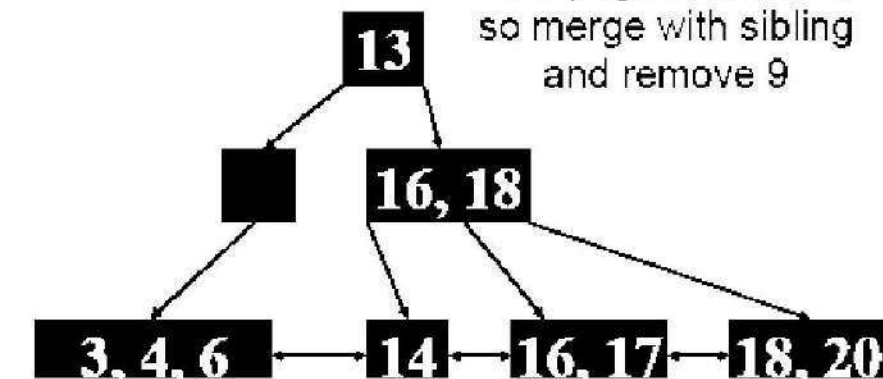
B+ Tree Deletion Example

Remove 9



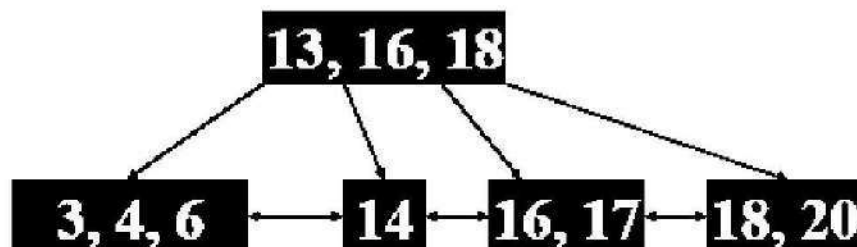
B+ Tree Deletion Example

Leaf page underflow,
so merge with sibling
and remove 9



B+ Tree Deletion Example

Index page underflow,
so merge with sibling
and demote 13

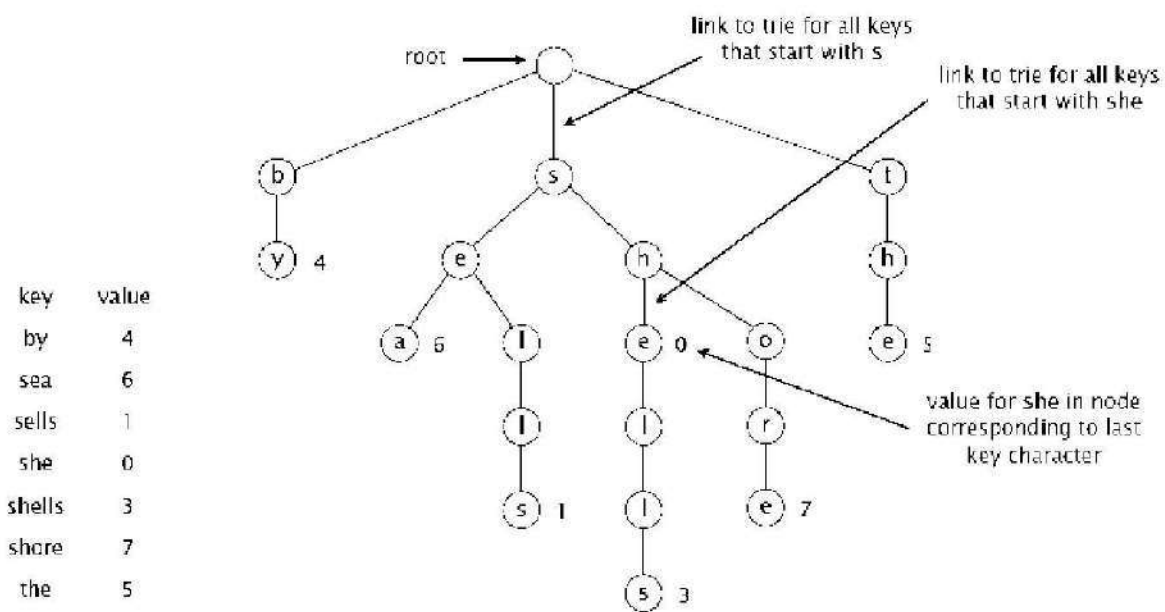


8. Explain about the Trie trees? (May 15)

Tries [from retrieval, but pronounced "try"]

Store characters in nodes (not keys).

Each node has R children, one for each possible character.(for now, we do not draw null links)

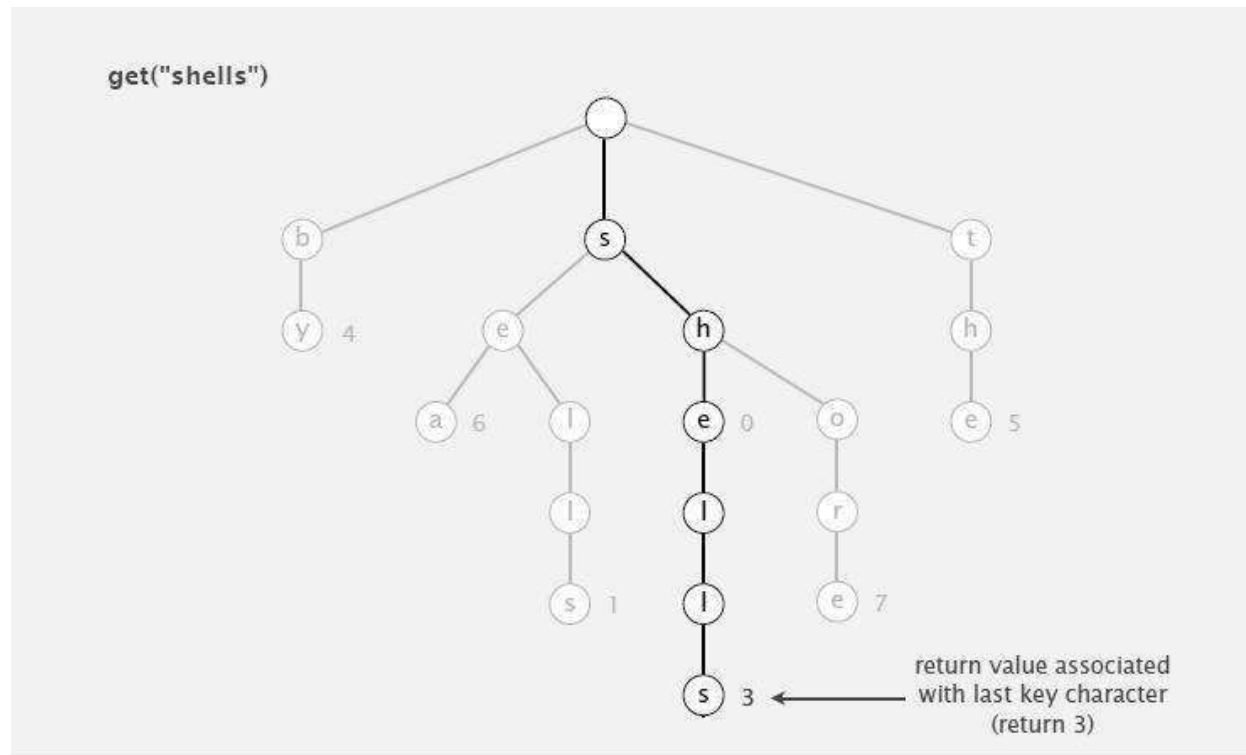


Search in a trie:

Follow links corresponding to each character in the key.

Search hit: node where search ends has a non-null value.

Search miss: reach null link or node where search ends has null value.

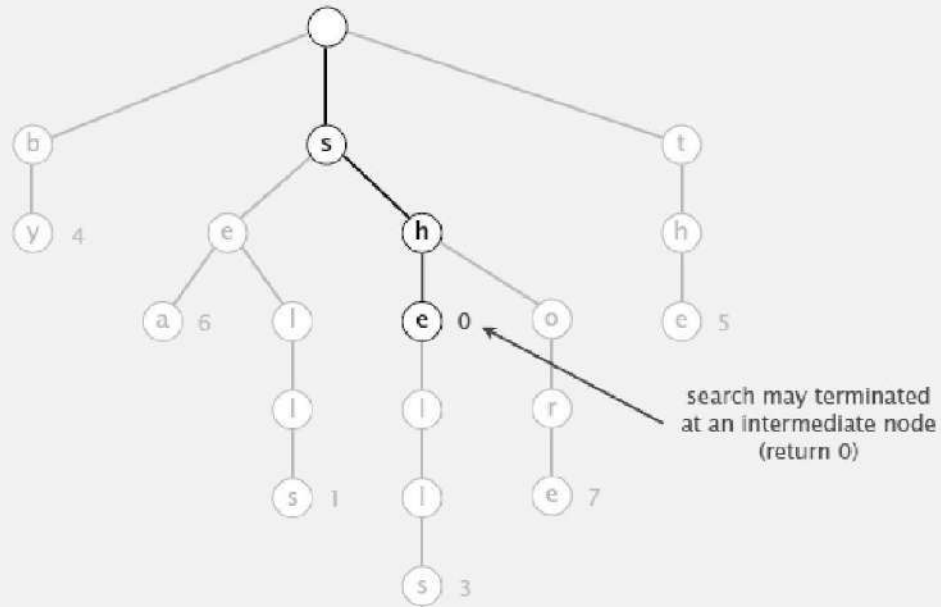


Follow links corresponding to each character in the key.

Search hit: node where search ends has a non-null value.

Search miss: reach null link or node where search ends has null value.

get("she")

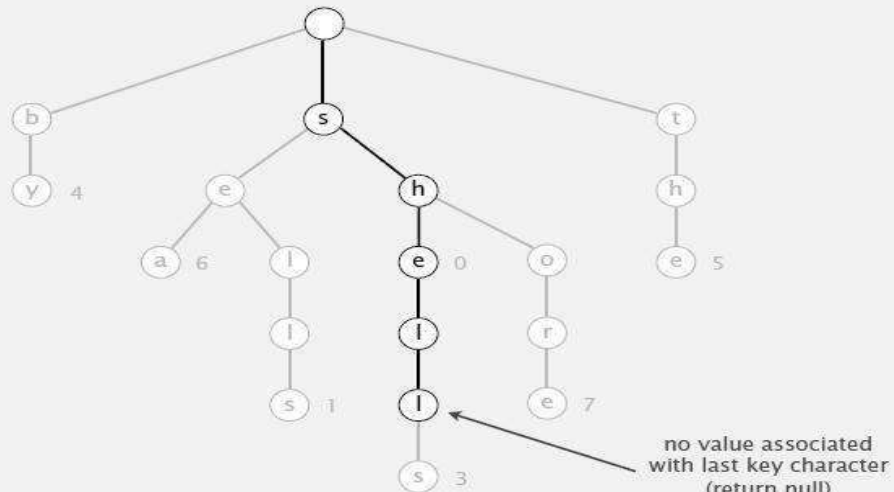


Follow links corresponding to each character in the key.

Search hit: node where search ends has a non-null value.

Search miss: reach null link or node where search ends has null value.

get("shell")



Follow links corresponding to each character in the key.

Search hit: node where search ends has a non-null value.

Search miss: reach null link or node where search ends has null value.

PONDICHERRY UNIVERSITY QUESTIONS

11 MARKS

NOV 2011(REGULAR)

1. Explain about the Binary tree traversal? (Pg. No. 10) (Qn. No. 2)
2. Explain about the B+Trees? (Pg. No. 28) (Qn. No. 7)

MAY 2012(ARREAR)

1. Explain about the AVL trees? (Pg. No. 15) (Qn. No. 4)
2. Explain about the B+Trees? (Pg. No. 28) (Qn. No. 7)

NOV 2012(REGULAR)

1. Explain about the Binary tree traversal? (Pg. No. 10) (Qn. No. 2)

MAY 2013(ARREAR)

1. Explain about the Binary tree traversal? (Pg. No. 10) (Qn. No. 2)
2. Explain about the B-trees? (Pg. No. 21) (Qn. No. 5)

MAY 2014(ARREAR)

1. Explain the traversals of binary tree with examples. (Pg. No. 10) (Qn. No. 2)
2. Describe the operation of B-tree (Pg. No. 21) (Qn. No. 5)
3. Describe in detail about B+ tree indexing (Pg. No. 28) (Qn. No. 7)

NOV 2014(REGULAR)

1. For the following list of elements create an AVL tree: 3,2,1,4,5,6,7,16,15,14 and perform the deletion of 15, 7 and 4.
2. Discuss the different methods of traversing a binary tree with algorithm. (Pg. No. 10) (Qn. No. 2)
3. What is a B tree? Write an algorithm to insert an element into the B-tree. (Pg. No. 21) (Qn. No. 5)
4. Illustrate the deletion operation on B+ tree and explain. (Pg. No. 28) (Qn. No. 7)

MAY 2015(ARREAR)

1. Explain tree traversals with examples. (Pg. No. 10) (Qn. No. 2)
2. Insert the numbers into a binary search tree: 8, 5,10,15,20,18,3
3. What is tries? Give some of the applications of tries. (Pg. No. 32) (Qn. No. 8)
4. Explain the rotations of AVL tree. (Pg. No. 15) (Qn. No. 4)

NOV 2015(REGULAR)

1. Give notes on (a.) Tree Traversals. (b.) Binary search Tree.
2. Explain the B+ tree indexing and trie tree indexing.

MAY 2016 (ARREAR)

1. Define Binary search tree.Explain its various operations.
2. Explain B+Tree Indexing with a an example.

DEC 2016(REGULAR)

- 1.Explain Binary Tree Traversal Algorithms.
- 2.Discuss in detail about shortest path algorithms.

